

Kubernetes v1.28.2 集群部署手册

环境说明

- 操作系统: CentOS Stream 9
- Kubernetes 版本: v1.28.2
- 容器运行时: containerd (v2.2.3)
- 网络插件: Flannel
- Pod CIDR: 10.244.0.0/16

服务器规划

角色	IP地址	主机名
Master	192.168.56.110	node1
Worker	192.168.56.120	node2
Worker	192.168.56.130	node3

第一部分：所有节点通用配置

1.1.基础环境配置

```
# 设置主机名（根据节点角色执行）
# node1 上执行
hostnamectl set-hostname node1
# node2 上执行
hostnamectl set-hostname node2
# node3 上执行
hostnamectl set-hostname node3
# 配置 hosts 文件（有节点执行），配置 IP 和主机名的映射
cat >> /etc/hosts <<EOF
192.168.56.110 node1
192.168.56.120 node2
192.168.56.130 node3
EOF
```

```
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6

192.168.56.110 node1
192.168.56.120 node2
192.168.56.130 node3

~
~
~
```

已经配置完成，每台节点都要配置上述内容

1.2.禁用 SWAP

所有节点执行

swapoff -a

sed -i '/ swap / s/^/#/' /etc/fstab

```
[root@node1 ~]# swapoff -a
[root@node1 ~]# sed -i '/swap/s/^/#/' /etc/fstab
[root@node1 ~]#
```

1.3 配置内核参数

所有节点执行

cat > /etc/sysctl.d/k8s.conf <<EOF

net.bridge.bridge-nf-call-ip6tables = 1

net.bridge.bridge-nf-call-iptables = 1

net.ipv4.ip_forward = 1

EOF

sysctl -system

```
[root@node2 ~]# cat > /etc/sysctl.d/k8s.conf <<EOF
> net.bridge.bridge-nf-call-ip6tables = 1
> net.bridge.bridge-nf-call-iptables = 1
> net.ipv4.ip_forward = 1
> EOF
[root@node2 ~]# sysctl --system
* Applying /usr/lib/sysctl.d/10-default-yama-scope.conf ...
* Applying /usr/lib/sysctl.d/50-coredump.conf ...
* Applying /usr/lib/sysctl.d/50-default.conf ...
* Applying /usr/lib/sysctl.d/50-libkcap-optmem_max.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /usr/lib/sysctl.d/50-redhat.conf ...
* Applying /etc/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.d/k8s.conf ...
* Applying /etc/sysctl.conf ...
kernel.yama.ptrace_scope = 0
```

1.4 加载内核模块

所有节点执行

cat > /etc/modules-load.d/containerd.conf <<EOF

overlay

br_netfilter

EOF

modprobe overlay

modprobe br_netfilter

```
[root@node2 ~]# # 3. 加载内核模块
[root@node2 ~]# modprobe overlay
[root@node2 ~]# modprobe br_netfilter
[root@node2 ~]#
```

1.5 关闭防火墙（或开放必要端口）

所有节点执行

systemctl stop firewalld

systemctl disable firewalld

```
[root@node1 ~]# systemctl status firewalld
○ firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; disabled; preset: enabled)
   Active: inactive (dead)
     Docs: man:firewalld(1)
[root@node1 ~]#
```

1.6.关闭 SELinux，三台主机都要关闭

```
[root@node1 ~]# setenforce 0
setenforce: SELinux is disabled
[root@node1 ~]# sed -i 's/^SELINUX=enforcing/SELINUX=disabled/' /etc/selinux/config
```

先使用 **setenforce 0** 检查 SELinux 是否关闭，出现 **disabled**，说明已经关闭

若没有关闭, 输入 **sed -i 's/^SELINUX=enforcing/SELINUX=disabled/' /etc/selinux/config** 来关闭 SELinux

1.7 时间同步，三个节点都要检查

```
[root@node1 ~]# systemctl status chronyd
● chronyd.service - NTP client/server
   Loaded: loaded (/usr/lib/systemd/system/chronyd.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-05-08 15:52:59 CST; 4h 12min ago
     Docs: man:chronyd(8)
           man:chrony.conf(5)
  Main PID: 831 (chronyd)
    Tasks: 1 (limit: 22966)
   Memory: 2.0M (peak: 2.5M)
      CPU: 154ms
   CGroup: /system.slice/chronyd.service
           └─831 /usr/sbin/chronyd -F 2

5月 08 15:55:19 node1 chronyd[831]: Detected falseticker 84.16.67.12 (2.centos.pool.ntp.org)
5月 08 15:55:19 node1 chronyd[831]: Selected source 193.182.111.142 (2.centos.pool.ntp.org)
5月 08 15:55:19 node1 chronyd[831]: System clock wrong by -1.058540 seconds
5月 08 15:55:20 node1 chronyd[831]: Detected falseticker 84.16.67.12 (2.centos.pool.ntp.org)
5月 08 15:56:23 node1 chronyd[831]: Selected source 210.28.130.4 (2.centos.pool.ntp.org)
5月 08 15:56:24 node1 chronyd[831]: Detected falseticker 139.199.214.202 (2.centos.pool.ntp.org)
5月 08 18:11:59 node1 chronyd[831]: Forward time jump detected!
5月 08 18:11:59 node1 chronyd[831]: Can't synchronise: no selectable sources (0 unreachable sources)
5月 08 18:15:13 node1 chronyd[831]: Selected source 139.199.214.202 (2.centos.pool.ntp.org)
5月 08 18:16:19 node1 chronyd[831]: Selected source 210.28.130.4 (2.centos.pool.ntp.org)
[root@node1 ~]#
```

Detected falseticker: 系统认为我使用的 NTP 服务器是“虚假服务器”，拒绝使用

Can't synchronise: no selectable sources: 没有可用的同步源，导致时间无法校准
做如下操作，完成时间同步

```

[root@node2 ~]#
[root@node2 ~]# # 备份原配置
[root@node2 ~]# cp /etc/chrony.conf /etc/chrony.conf.bak
[root@node2 ~]#
[root@node2 ~]# # 写入国内 NTP 源
[root@node2 ~]# cat > /etc/chrony.conf <<EOF
> pool ntp.aliyun.com iburst
> pool time.windows.com iburst
> pool ntp.tencent.com iburst
> driftfile /var/lib/chrony/drift
> makestep 1.0 3
> rtcsync
> allow 192.168.0.0/16
> keyfile /etc/chrony.keys
> leapsectz right/UTC
> logdir /var/log/chrony
> EOF

```

重启 chroyd 服务

```

[root@node2 ~]# systemctl restart chronyd
[root@node2 ~]# chronyc makestep
200 OK
[root@node2 ~]#
[root@node2 ~]# chronyc tracking
Reference ID      : 6A37B8C7 (106.55.184.199)
Stratum          : 3
Ref time (UTC)   : Fri May 08 12:09:26 2026
System time      : 0.000000000 seconds slow of NTP time
Last offset      : +0.003813223 seconds
RMS offset       : 0.003813223 seconds
Frequency        : 11.096 ppm slow
Residual freq    : -3371.264 ppm
Skew             : 1.284 ppm
Root delay       : 0.169450969 seconds
Root dispersion  : 0.095985599 seconds
Update interval  : 1.9 seconds
Leap status      : Normal

```

看到 Leap status: Normal 就说明同步成功了

第二部分：安装配置容器运行时（containerd）

2.1 安装 containerd

所有节点执行

```
yum install -y containerd.io
```

2.2 配置 containerd（关键步骤）

所有节点执行

```
mkdir -p /etc/containerd
```

生成默认配置

```
containerd config default > /etc/containerd/config.toml
```

修改 cgroup 驱动为 systemd

```
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

# 修改 sandbox 镜像为阿里云镜像（解决国内无法访问问题）
sed -i 's|registry.k8s.io/pause:3.9|registry.aliyuncs.com/google_containers/pause:3.9|g'
/etc/containerd/config.toml
```

2.3 创建 containerd 服务文件（如果没有）

```
# 所有节点执行
cat > /etc/systemd/system/containerd.service <<EOF
[Unit]
Description=containerd container runtime
Documentation=https://containerd.io
After=network.target local-fs.target

[Service]
ExecStartPre=-/sbin/modprobe overlay
ExecStart=/usr/bin/containerd
Type=notify
Delegate=yes
KillMode=process
Restart=always
RestartSec=5
LimitNPROC=infinity
LimitCORE=infinity
LimitNOFILE=infinity
TasksMax=infinity
OOMScoreAdjust=-999

[Install]
WantedBy=multi-user.target
EOF

# 启动 containerd
systemctl daemon-reload
systemctl start containerd
systemctl enable containerd
```

2.4 手动拉取 pause 镜像

```
# 所有节点执行（避免集群初始化时拉取失败）
ctr image pull registry.aliyuncs.com/google_containers/pause:3.9
ctr image tag registry.aliyuncs.com/google_containers/pause:3.9 registry.k8s.io/pause:3.9
```



```

[root@node2 ~]# kubectl image pull registry.aliyuncs.com/google_containers/pause:3.9
WARN[000] DEPRECATION: The "mirrors" property of "[plugins.io.containers.grpc.v1.cri].registry" is deprecated since containerd v1.5 and will be removed in containerd v2.1.Use "config.p
th" instead.
registry.aliyuncs.com/google_containers/
├──index (7031c1b2839)
│   ├──manifest (461c1e05779)
│   │   └──config (1cf778a09da73)
│   ├──manifest (3ec98b8452dc)
│   │   └──config (829e3d6d38bd)
│   ├──manifest (8d410e686c9)
│   │   └──config (e6f181638937)
│   ├──manifest (cad97be0e860)
│   │   └──config (b1a439917a2)
│   ├──manifest (d13e24f4bd9)
│   │   └──config (f393c7d2b9c3)
│   ├──manifest (4e2a8276c1b)
│   │   └──config (ad8a2f65cab)
│   └──manifest (c08153e6dc7)
│       └──config (83514219e66)
application/vnd.docker.distribution.manifest.list.v2+json sha256:7031c1b283838d2e09657bacb03c05ebd362dc88d64b46cc4772a21097
Pulling from OCI Registry (registry.aliyuncs.com/google_containers/pause:3.9)
[0.0s] # kubectl image tag registry.aliyuncs.com/google_containers/pause:3.9 registry.k8s.io/pause:3.9
WARN[000] DEPRECATION: The "mirrors" property of "[plugins.io.containers.grpc.v1.cri].registry" is deprecated since containerd v1.5 and will be removed in containerd v2.1.Use "config.p
th" instead.
registry.k8s.io/pause:3.9

[root@node2 ~]# kubectl image tag registry.aliyuncs.com/google_containers/pause:3.9 registry.k8s.io/pause:3.9
WARN[000] DEPRECATION: The "mirrors" property of "[plugins.io.containers.grpc.v1.cri].registry" is deprecated since containerd v1.5 and will be removed in containerd v2.1.Use "config.p
th" instead.
registry.k8s.io/pause:3.9

[root@node2 ~]# kubectl run --image=registry.k8s.io/pause:3.9 --name=containerd.sock --restart=Never --command=ls /run/containerd/containerd.sock
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with "kubectl -n kube-system get cm kubeadm-config -o yaml"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
 * Certificate signing request was sent to applier and a response was received.
 * The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

```

第三部分：安装 Kubernetes 组件

3.1 添加 Kubernetes YUM 源

```
# 所有节点执行
```

```
cat > /etc/yum.repos.d/kubernetes.repo <<EOF
```

[kubernetes]

name=Kubernetes

```
baseurl=https://mirrors.aliyun.com/kubernetes/yum/repos/kubernetes-el7-x86_64/
```

```
enabled=1
```

gpgcheck=1

```
repo_gpgcheck=1
```

```
gpgkey=https://mirrors.aliyun.com/kubernetes/yum/doc/yum-key.gpg
```

<https://mirrors.aliyun.com/kubernetes/yum/doc/rpm-package-key.gpg>

EOF

3.2 安装 kubeadm、kubectl、kubeadm

```
# 所有节点执行
```

```
yum install -y kubelet-1.28.2 kubeadm-1.28.2 kubectl-1.28.2
```

```
# 启动 kubelet
```

```
systemctl enable kubelet
```

```
systemctl start kubelet
```

```
[root@node1 k8s]# dnf install -y kubeadm kubelet
```

Repository epel is listed more than once in the configuration
本次元数据批量检查: 0:59:20 前, 执行于: 2025年05月08日 星期五 20:09:19分19秒。
依赖关系解决。

软件包	架构	版本	仓库	大小
安装:				
kubeadm	x86_64	1.28.2-0	kubernetes	11 M
kubelet	x86_64	1.28.2-0	kubernetes	21 M
安装依赖关系:				
cri-tools	x86_64	1.26.0-0	kubernetes	8.6 M
kubectx	x86_64	1.28.2-0	kubernetes	11 M
kubernetes-cni	x86_64	1.2.0-0	kubernetes	17 M
sscat	x86_64	1.7.4-1-8.el9	appstream	303 K

事务概要

三个节点都执行 **dnf install -y kubeadm kubelet**

三个节点都自启动 kubelet

```
[root@node1 k8s]# systemctl enable kubelet
Created symlink /etc/systemd/system/multi-user.target.wants/kubelet.service → /usr/lib/systemd/system/kubelet.service.
[root@node1 k8s]#
```

第四部分：Master 节点初始化

4.1 初始化集群

仅在 node1 (Master) 上执行

kubeadm init \

--apiserver-advertise-address=192.168.56.110 \

--image-repository=registry.aliyuncs.com/google_containers \

--kubernetes-version=v1.28.2 \

--pod-network-cidr=10.244.0.0/16 \

--cri-socket=unix:///run/containerd/containerd.sock

```
[root@node1 ~]# kubeadm init \
> --apiserver-advertise-address=192.168.56.110 \
> --image-repository=registry.aliyuncs.com/google_containers \
> --kubernetes-version=v1.28.2 \
> --pod-network-cidr=10.244.0.0/16 \
> --cri-socket=unix:///run/containerd/containerd.sock
```

初始化过程中，出现下列内容时，说明初始化成功

```
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

4.2 配置 kubectl

仅在 Master 上执行

mkdir -p \$HOME/.kube

cp -i /etc/kubernetes/admin.conf \$HOME/.kube/config

chown \$(id -u):\$(id -g) \$HOME/.kube/config

4.3 验证 Master 节点状态

查看节点状态（此时应该是 NotReady，等待安装网络插件）

kubectl get nodes

```
[root@node1 ~]# kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
node1       Ready     control-plane  2m3s  v1.28.2
```

第五部分：安装网络插件（Flannel）

5.1 创建 Flannel ConfigMap

在 Master 上执行

kubectl apply -f - <<EOF

apiVersion: v1

kind: ConfigMap

metadata:

name: kube-flannel-cfg

namespace: kube-flannel

data:

net-conf.json: |

```
{
  "Network": "10.244.0.0/16",
  "Backend": {
    "Type": "vxlan"
  }
}
```

cni-conf.json: |

```
{
  "name": "cbr0",
  "cniVersion": "0.3.1",
  "plugins": [
    {
      "type": "flannel",
      "delegate": {
        "hairpinMode": true,
        "isDefaultGateway": true
      }
    },
    {
      "type": "portmap",
      "capabilities": {
```



```

        "portMappings": true
      }
    }
  ]
}
EOF

```

5.2 部署 Flannel（简化版，使用 pause 镜像）

```

# 在 Master 上执行
kubectl apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: kube-flannel
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: flannel
  namespace: kube-flannel
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: flannel
rules:
- apiGroups: ['']
  resources: ['pods']
  verbs: ['get', 'list', 'watch']
- apiGroups: ['']
  resources: ['nodes']
  verbs: ['get', 'list', 'watch']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: flannel
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: flannel
subjects:
- kind: ServiceAccount
  name: flannel

```

```

    namespace: kube-flannel
---
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: kube-flannel-ds
  namespace: kube-flannel
spec:
  selector:
    matchLabels:
      app: flannel
  template:
    metadata:
      labels:
        app: flannel
    spec:
      hostNetwork: true
      serviceAccountName: flannel
      tolerations:
        - operator: Exists
      containers:
        - name: pause
          image: registry.aliyuncs.com/google_containers/pause:3.9
          imagePullPolicy: IfNotPresent
          command: ["/pause"]
          securityContext:
            privileged: true
          volumeMounts:
            - name: flannel-cfg
              mountPath: /etc/kube-flannel
      volumes:
        - name: flannel-cfg
          configMap:
            name: kube-flannel-cfg

```

EOF

5.3 验证网络插件

查看 Flannel Pod 状态

kubectl get pods -n kube-flannel

```

[root@node1 ~]# # 安装 Flannel 网络插件
[root@node1 ~]# kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created

```

等待所有 Pod 变为 Running (约 30 秒)

kubectl get nodes

```
[root@node1 ~]# # 查看 Flannel Pod 状态
[root@node1 ~]# kubectl get pods -n kube-flannel -w
NAME                                READY    STATUS    RESTARTS   AGE
kube-flannel-ds-6qfrz               0/1     Init:1/2   0           56s
kube-flannel-ds-ng22x               0/1     Init:0/2   0           56s
kube-flannel-ds-q2kd2               0/1     Init:1/2   0           56s
```

第六部分：Worker 节点加入集群

6.1 在 Master 上生成 Join 命令

在 Master 上执行

kubeadm token create --print-join-command

如下所示

```
[root@node1 ~]# kubeadm token create --print-join-command
kubeadm join 192.168.56.110:6443 --token 4ndbrg.1q7kcz94oawku7 --discovery-token-ca-cert-hash sha256:6b8228e4c62e1b279be60d255379526769dce413ef3874c0fb0f5bb6e3af61d1 --cr
```

6.2 Worker 节点加入集群

在 node2 和 node3 上执行

kubeadm join 192.168.56.110:6443 --token <你的 token> --discovery-token-ca-cert-hash sha256:<你的 hash> --cri-socket=unix:///run/containerd/containerd.sock

例如 node2 会出现下列内容，说明成功加入集群

```
[root@node2 ~]# kubeadm join 192.168.56.110:6443 --token 4ndbrg.1q7kcz94oawku7 --discovery-token-ca-cert-hash sha256:6b8228e4c62e1b279be60d255379526769dce413ef3874c0fb0f5bb6e3af61d1 --cri-socket=unix:///run/containerd/containerd.sock
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control plane to see this node join the cluster.
```

6.3 在 Worker 节点上手动创建 CNI 配置（如需要）

如果节点加入后一直 NotReady，在 Worker 节点上执行

mkdir -p /etc/cni/net.d

cat > /etc/cni/net.d/10-flannel.conflist <<EOF

```
{
  "name": "cbr0",
  "cniVersion": "0.3.1",
  "plugins": [
    {
      "type": "flannel",
      "delegate": {
        "hairpinMode": true,
        "isDefaultGateway": true
      }
    }
  ],
}
```

```

    {
      "type": "portmap",
      "capabilities": {
        "portMappings": true
      }
    }
  ]
}
EOF
mkdir -p /run/flannel
cat > /run/flannel/subnet.env <<EOF
FLANNEL_NETWORK=10.244.0.0/16
FLANNEL_SUBNET=10.244.1.1/24
FLANNEL_MTU=1450
FLANNEL_IPMASQ=true
EOF
systemctl restart kubelet

```

第七部分：集群验证

7.1 查看节点状态

在 Master 上执行

```
#kubectl get nodes
```

```

[root@node1 ~]# sleep 30
[root@node1 ~]# kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
node1       Ready     control-plane   23m   v1.28.2
node2       Ready     <none>        19m   v1.28.2
node3       Ready     <none>        19m   v1.28.2
[root@node1 ~]#

```

所有节点都在 **Ready** 状态，集群部署成功

7.2 查看所有系统 Pod

```
#kubectl get pods -n kube-system
```

```

[root@node1 ~]# # 查看所有系统 Pod (应该全部 Running)
[root@node1 ~]# kubectl get pods -n kube-system
NAME                                READY   STATUS    RESTARTS   AGE
coredns-66f779496c-9kmxr            1/1     Running   0           24m
coredns-66f779496c-9txwb            1/1     Running   0           24m
etcd-node1                          1/1     Running   3           24m
kube-apiserver-node1                 1/1     Running   3           24m
kube-controller-manager-node1        1/1     Running   3           24m
kube-proxy-268fq                     1/1     Running   0           20m
kube-proxy-6vzdx                     1/1     Running   0           20m
kube-proxy-z97kx                     1/1     Running   0           24m
kube-scheduler-node1                 1/1     Running   3           24m
[root@node1 ~]#

```

常见问题及解决方案

问题 1: kubeadm init 卡在 wait-control-plane

原因: containerd 无法拉取 pause 镜像

解决:

修改 containerd 配置

```
sed -i 's|registry.k8s.io/pause:3.10|registry.aliyuncs.com/google_containers/pause:3.9|g' /etc/containerd/config.toml
systemctl restart containerd
```

手动拉取镜像

```
ctr image pull registry.aliyuncs.com/google_containers/pause:3.9
ctr image tag registry.aliyuncs.com/google_containers/pause:3.9 registry.k8s.io/pause:3.9
```

问题 2: Flannel Pod 卡在 Init:0/2 或 Init:1/2

原因: 无法拉取 Flannel 镜像

解决: 使用简化版 Flannel (只依赖 pause 镜像), 见第五部分

问题 3: 节点一直 NotReady

原因: CNI 配置问题

解决:

在 Worker 节点上手动创建 CNI 配置

```
mkdir -p /etc/cni/net.d
```

```
cat > /etc/cni/net.d/10-flannel.conflist <<'EOF'
```

```
{
  "name": "cbr0",
  "cniVersion": "0.3.1",
  "plugins": [
    {"type": "flannel", "delegate": {"hairpinMode": true, "isDefaultGateway": true}},
    {"type": "portmap", "capabilities": {"portMappings": true}}
  ]
}
EOF
```

```
systemctl restart kubelet
```